



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich



Reliability and Risk Engineering

Louisa Hillegaat

Graph Neural Network solver for Power Flow

Semester Thesis

RRE – Reliability and Risk Engineering Lab
Swiss Federal Institute of Technology (ETH) Zurich

Expert: Prof. Dr. Giovanni Sansavini
Supervisors: Anna Varbella

Zürich, January 6, 2023

Abstract

In this thesis, we are looking to find a novel solution to the Power Flow equations. We focused our research on incorporating the physics of Power Systems into Graph Neural Networks. Firstly we give a brief introduction to Power Systems and Load study by deriving the Power Flow equations in a node network. Secondly, we will explain the concepts of graphs, more importantly, their use and the fundamental terms used. Then we highlight the need for novel machine-learning methods that can incorporate graph data which introduces the notion of Graph Neural Networks. We will focus on the message-passing layer aspect of Graph Neural Networks. Lastly, we will present our method using Graph Neural Network to solve the Power Flow equations for a 24-Bus system.

Contents

1	Introduction	1
2	Power System Analysis	3
2.1	Introduction to Power Systems Analysis	3
2.1.1	Node network	5
2.1.2	Power flow computation	6
2.2	Approximative DC Solutions to the Power Flow Problem	9
3	Graph Neural Networks	11
3.1	Graph Topology	11
3.2	Neural Networks	12
3.2.1	Fully Connected Neural Network	13
3.2.2	Convolutional Neural Networks	14
3.3	Graph Neural Network	14
3.3.1	Message passing layers	15
4	Graph Neural Network for Power Flow	19
4.1	Traditional method	20
4.2	Power Flow computation with GNN	22
4.2.1	Dataset	22
4.2.2	Model Architecture	24
4.3	Results	25
4.3.1	Loss and Predictions	25
4.3.2	Computation time	28
5	Conclusions and Outlook	31
	Bibliography	32

List of Figures

1.1	Electrical Grid	2
2.1	Lumped-circuit model (π -model) of a transmission line between node k and m.	4
2.2	Network node types [1]	6
2.3	Generic bus with sign conventions from current and power flows [1]	7
3.1	A graph for structure-based predictions of protein-protein interactions [2] and a social network [3]	12
3.2	Fully connected Neural network [4]	13
3.3	Convolutional Neural Network [5]	14
3.4	Input graph and computation graph for a target node [6]	15
3.5	Attention mechanism(left) and multi-head attention with K=3 heads by node 1 of its neighborhood. Different arrow styles show different computation methods [7]	17
4.1	Graph representation of a 24-Bus electrical power grid	20
4.2	Model Architecture: Two GAT layers and two FCNN layers	24
4.3	Graph of train loss (logarithmic scale) for GNN model	26
4.4	Predicted against real output values for hour 1	27
4.5	Predicted against real output values for hour 9	27
4.6	Predicted against real output values for hour 28	28
4.7	Batch size, computation time and test loss for predictions with GNN model	29
4.8	Computation time with Matpower	29

Chapter 1

Introduction

The Electrical Power System is considered one of the most important and complex infrastructures ever built by humans. It is crucial to constantly develop this infrastructure, to make it more resilient, more efficient, and sustainable [1]. Especially today, with the geopolitical conflicts, climate crisis, and resource scarceness that govern the world ensuring the dependability of the electric energy supply while conserving the environment is crucial.

Electrical Power Systems began as isolated systems and progressively expanded to provide for entire countries and even continents. Hence, they were mostly treated as stand-alone units. A result of greater interconnection is a more efficient and sustainable use of power as fewer generators are needed. It also has a great economic advantage, for example, it can be less expensive to purchase some electricity from another supplier than to use its own supply. Electricity is now routinely traded across the systems of various utilities. Many new problems rose with interconnected systems as the amount of current flowing has largely increased. The disruption brought on by a short circuit on one system might spread to other systems that are linked to it. This means proper stability and resilience mechanism (protective relays and circuit breakers) need to be implemented to prevent failure of the complete electrical grid. Determining how the total generation needed at any one moment should be split across the various plants and among the units within each plant is a significant issue. Load studies, fault calculations, designs for safeguarding the system from lightning and switching surges and against any short circuit, as well as studies on the system's stability, are necessary for planning the operation, improvement, and development of a Power System.

In this thesis, we will focus on Load study. Load study is known as the measurement of voltage, current power, reactive power, and power factor at various locations in an electrical network under specific circumstances. It is essential as it determines the stability of the network when for example new loads, new generating stations, and new transmission lines are installed. Not only it is used to compute the "normal state" (when they are no outages or disturbances) but it is also very important for ensuring safety in an N-1 contingency case (which is the state called when there is an outage). For example, if a transmission line breaks it would lead to redistribution of the power flows

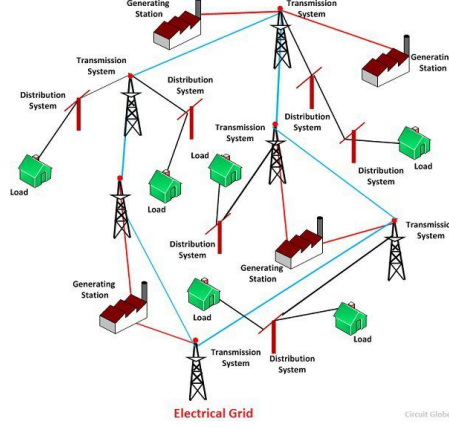


Figure 1.1: Electrical Grid

and would cause overloading. A fast computation of the features of the nodes (Power, Voltage magnitude, Voltage angle) at any moment in time would make it easier to take action and design mechanisms to ensure the stability of the network. Load study lays on the basis of the power flow equations which are derived from the Kirchhoff and Ohms Law. The Power flow equations are a representation of the physical relationships between voltages, currents, power injections, and loads. Solving these equations for a large network with traditional methods such as the Newton-Raphson method [8] is possible but often computationally expensive. Therefore, it is most important to find solvers for these equations that are computationally less invasive and take an acceptable amount of time.

In this thesis, we are evaluating the adequacy of a Graph Neural Network as a solver for the Power Flow equations. In the first section, we will introduce the fundamentals of Power System Analysis and derive the power flow equations from Kirchhoff and Ohms Laws. Secondly, we will introduce Graph Neural network and explain machine learning with graphs. Lastly, we have implemented a Graph Neural network for an electrical grid (24 buses) and found solutions to the Power flow equations. We have evaluated this novel power flow solver and compared it to traditional solver solutions.

Chapter 2

Power System Analysis

In this chapter, we will derive the power flow equations and discuss various models in how this problem is described. The models will be used in the subsequent chapters as a formulation for the power flow problem and serve as a description of interactions between various parameters (Current, Voltage, Power, Impedance) at a steady state. Originally, the voltage and current on an electrical transmission line were described by a pair of connected, linear partial differential equations known as the telegrapher's equations (or simply the telegraph equations) [9]. The solution to these equations with a focus on the conditions at the end of the transmission lines gives an algebraic model: the lumped-circuit model π -models [10]. While being the most accurate model, it is computationally very expensive and most of the information computed is not needed. Other solutions have been derived with simpler models, that are computationally less invasive and need less reliable data input which can be hard to obtain. In the subsequent chapter, we will derive a solution to the steady-state power flow equations based on the physical constraints of a power system. The Power Systems Analysis chapter of this thesis is largely based on the script from Gabriela Hug "Power System Analysis" from ETH Zürich, 2021.

2.1 Introduction to Power Systems Analysis

In Fig. 2.1, we have depicted a Lumped-circuit model π model for symmetrical steady-state conditions. This figure is derived from the telegraph equation and is described by the following parameters:

- R_{km} = Series resistance Ω/km
- X_{km} = Series reactance Ω/km
- $B_{km} = -\frac{X_{km}}{R_{km}^2 + X_{km}^2}$ Shunt susceptance (siemens/km)
- $G_{km} = \frac{R_{km}}{R_{km}^2 + X_{km}^2}$ Shunt conductance (siemens/km)

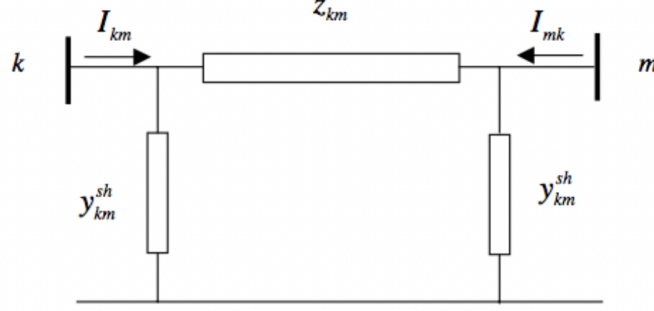


Figure 2.1: Lumped-circuit model (π -model) of a transmission line between node k and m.

- θ_{km} = Angle between bus k and m

When we have a transmission line that spans from bus 1 to bus 2, this transmission line has an impedance. To model this impedance we have to have a resistance component and a reactive component. The impedance Z and the admittance Y shown in Figure 2.1 are described as follows:

$$Z_{km} = R_{km} + jX_{km} \quad (2.1)$$

$$Y_{km}^{sh} = 1/Z_{km} = G_{km}^{sh} + jB_{km}^{sh} \quad (2.2)$$

The shunt element depicted in Fig.(2.1) is used to increase the power and energy system efficiency as it absorbs and compensates the reactive power in cables and long high voltage transmission lines [11]. In short, it controls the voltage and continuously adjusts the compensation according to the load variation. Therefore the shunt element should be considered in power flow computation. Intuitively, admittance is a measure of how easy the current is allowed to travel through the transmission line. We define the current going from bus k to bus m by using Ohm's law [12]:

$$I_{km} = \frac{E_k - E_m}{Z_{km}} + \frac{E_k}{Z_{km}^{sh}} = \frac{E_{km}}{Z_{km}} + \frac{E_k}{Z_{km}^{sh}} = Y_{km} \cdot E_{km} + Y_{km}^{sh} E_k \quad (2.3)$$

where E_k is the voltage at bus k and θ_k the angle at bus k. This can be written in matrix form such that:

$$\begin{pmatrix} I_{km} \\ I_{mk} \end{pmatrix} = \begin{pmatrix} Y_{km} + Y_{km}^{sh} & -Y_{km} \\ -Y_{km} & Y_{km} + Y_{km}^{sh} \end{pmatrix} \begin{pmatrix} E_k \\ E_m \end{pmatrix} \quad (2.4)$$

The matrix on the right-hand side of Eq.(2.4) is symmetric and since the diagonal elements are equal it shows that the lines and cables are a symmetrical element. The complex voltage is described as follows:

$$E_k = U_k e^{j\theta_k} \quad (2.5)$$

As seen in Fig. 2.1, the current from a shunt is defined as positive when injected into a bus. For sign convention, we can therefore write :

$$I_k^{sh} = -Y_k^{sh} E_k \quad (2.6)$$

With the current and the voltage we can also define the complex power, which is given by :

$$\begin{aligned} S_{km} &= P_k + jQ_k = E_k I_{km}^* + S_{km}^{sh} \\ &= E_k Y_{km}^* E_{km}^* - (Y_{km}^{sh})^* |E_k|^2 = Y_{km}^* U_k e^{j\theta_k} (U_k e^{j\theta_k} - U_m e^{j\theta_m}) - (Y_{km}^{sh})^* |E_k|^2 \\ &= (G_{km} - jB_{km}) U_k (\cos(\theta_k) + j \sin(\theta_k)) (U_k e^{j\theta_k} - U_m e^{j\theta_m}) - jB_{km}^{sh} U_k^2 \end{aligned} \quad (2.7)$$

with U_k the voltage magnitude at node k and we neglect the conductance of Y_k^{sh} , i.e $G_k^{sh} = 0$. Further computation and separation of real and complex values give the following power flow equations for transmission lines :

$$P_{km} = U_k^2 G_{km} - U_k U_m (G_{km} \cos(\theta_{km}) - B_{km} \sin(\theta_{km})) \quad (2.8)$$

$$Q_{km} = -U_k^2 (B_{km} + B_{km}^{sh}) - U_k U_m (G_{km} \sin(\theta_{km}) - B_{km} \cos(\theta_{km})) \quad (2.9)$$

From Eq.(2.8) and Eq.(2.19) we can see that the shunt element only has an effect on the reactive power.

2.1.1 Node network

Load flow or Power flow computation gives an understanding of the steady-state behavior of power systems. It is a helpful computation as it can be used in various situations. For example, it is used in predicting the amount of power needed to generate a city, making sure that transmission lines are not overloaded, and giving insights into the net power flows. In brief, "Load flow computes the voltages in the network and visualizes the effect of tap-changing transformers, capacitor banks and load shedding on the voltage profile in the system" [1]. In reality, a so-called "contingency analysis" is performed, which is an analysis relying on a large number of load flow computations, to prevent a failure of the whole power grid (overload of transmission lines, transformers, etc.). The load flow computation is defined and constrained by the network topology, the network parameters, and information on the node. A network node k is described by four parameters $(U_k, \theta_k, P_k, Q_k)$. It outputs the node voltages, the transmission line Power and current, the power losses, the generator reactive power and the slack node power. We need to distinguish between different type of nodes in our network:

Type of Nodes

- **The load bus (PQ bus):** Buses without a linked generator are considered load buses. In place of impedances, loads are treated as constant power sinks where active and

Node Type	Number of Nodes	Specified	Unknown
Slack Node ($i = 1$)	1	U_i, θ_i	P_i, Q_i
PU Bus ($i = 2, \dots, N_g + 1$)	N_g	P_i, U_i	Q_i, θ_i
PQ Bus ($i = N_g + 2, \dots, N$)	$N - N_g - 1$	P_i, Q_i	U_i, φ_i

Figure 2.2: Network node types [1]

reactive power injections are included (for a load the injections are fixed). This means that the voltage's magnitude and step angle must be calculated using the load flow equation. Active and reactive power balance.

- **The generator bus (PU bus):** is the bus that connects the generator to its generating transformer. The real power and voltage are specified for buses that are generators. Reactive power injection is changed such that the voltage reaches the predetermined fixed value. Active power balance and fixed voltage magnitude.
- **The slack node:** (Fixed voltage magnitude and angle) In order to solve the equation in a Power System, we need to define a slack node. The slack node serves as a reference for the other nodes and it is the only node where the angle of the voltage is specified. In load flow computation all the unknown voltages at each bus need to be calculated before the current and the losses can be calculated. The total difference between the active total active power input and total active power output plus the computed I^2R losses is balanced by the slack node. Generally, the slack node must be assigned to a generator bus. It is common to take a voltage angle of $\theta = 0$. The slack node is a mathematical construct used for steady-state power flow computation, in reality, it is a dynamic system and all the generators at different nodes balance the losses.

The table in Fig.2.2 shows the network node types and an overview of the specified values that are known for each node. This will be important when integrating the data from the nodes into our GNN.

2.1.2 Power flow computation

Having explained the node network of a Power System we can now derive the Power Flow equations for each of the buses in a power grid. We can use the power flow equation from transmission lines derived in the previous chapter Eq.2.8 and Eq.2.9 and apply them to a node network. Per bus, we have two unknown variables: the voltage magnitude U_k and the voltage angle θ_k . The unknown variables are not always the same (depending on the type of node e.g. for PV nodes the voltage magnitude is already known and for slack bus the voltage magnitude and angle are already known). However, we introduce voltage magnitude and angle as variables for all buses. We have two variables per bus, so we need two equations per bus to have a well-determined system. At any bus the law of energy conversion holds meaning that injected power is equal to generated power, this

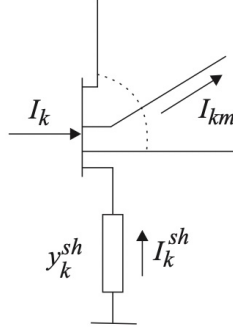


Figure 2.3: Generic bus with sign conventions from current and power flows [1]

holds for active and reactive power.

Using Kirchhoff's Current Law (KCL) we can define the current at the bus k . Kirchhoff's law states that the current injected has to be equal to the current going out of the node. In Fig.2.3 we have depicted a bus with current inflows and outflows from [1], this is useful for the sign convention chosen when applying KCL. From Eq. 2.3 we have:

$$I_k + I_k^{sh} = \sum_{m \in \Omega_k} I_{km} \quad \text{for } k=1, \dots, N \quad (2.10)$$

where Ω_k is the set of nodes adjacent to k (excluding bus k), I_k is the net current injections from generators and loads, and I_k^{sh} is the current injection from shunts, m is a node adjacent to k and N is the number of nodes in the network.

From 2.3 we can set up an expression such that 2.10 can be written in matrix form :

$$\mathbf{I} = \mathbf{Y}'\mathbf{E} \quad (2.11)$$

where

- $\mathbf{I}$ is the injection vector with elements I_k , $k=1, \dots, N$
- $\mathbf{E}$ is the nodal voltage vector $E_k = U_k e^{j\theta_k}$
- $\mathbf{Y}' = \mathbf{G} + j\mathbf{B}$ is the nodal admittance matrix

The admittance matrix is written:

$$Y' = \begin{pmatrix} Y'_{11} & \cdots & Y'_{1j} \\ \vdots & \ddots & \vdots \\ Y'_{i1} & \vdots & Y'_{ij} \end{pmatrix} \quad (2.12)$$

Note this is the nodal admittance matrix without transformers. $\mathbf{Y}$ is not necessarily symmetric for the case with transformers. The diagonal entries represent the admittance

at a specific bus and the off-diagonal entries the admittance of a transmission line. From 2.3 we can rewrite:

$$Y'_{km} = -Y_{km} \quad (2.13)$$

$$Y'_{kk} = Y_{km} + Y_{km}^{sh} \quad (2.14)$$

The k_{th} component of $\mathbf{I}$ defined, I_k can be written as such:

$$I_k = Y'_{kk} E_k + \sum_{m \in \Omega_k} Y'_{km} E_m = \sum_{m \in K} Y'_{km} E_m \quad (2.15)$$

where K is the set of buses adjacent to bus k , including bus k . Considering that we know that $Y'_{km} = G'_{km} + jB'_{km}$ and $E_m = U_m e^{j\theta_m}$ Eq. 2.15 can be rewritten to

$$I_k = \sum_{m \in K} (G'_{km} + jB'_{km}) (U_m e^{j\theta_m}) \quad (2.16)$$

and the complex power injection at bus k is defined such that :

$$S_k = E_k I_k^* = U_k e^{j\theta_k} \sum_{m \in K} (G'_{km} - jB'_{km}) (U_m e^{-j\theta_m}) \quad (2.17)$$

and by identifying the real and imaginary parts we obtain the total active and reactive power injected into the system:

$$P_k = P_{Gk} - P_{Lk} = U_k \sum_{m \in K} (G'_{km} \cos(\theta_{km}) + B'_{km} \sin(\theta_{km})) \quad (2.18)$$

$$Q_k = Q_{Gk} - Q_{Lk} = U_k \sum_{m \in K} U_m (G'_{km} \sin(\theta_{km}) - B'_{km} \cos(\theta_{km})) \quad (2.19)$$

Procedure to setup the power flow equations using nodal formulation

1. Set up admittance matrix
2. For each bus, identify the type of bus
3. Set up the power flow equation according to :
 - PQ Bus : active and reactive power balance using 2.18 and 2.19

$$\begin{aligned} P_k &= -P_{Lk} \\ Q_k &= -Q_{Lk} \end{aligned} \quad (2.20)$$

- PU bus: active power balance and fixed voltage from 2.18

$$\begin{aligned} P_k &= P_{Gk} - P_{Lk} \\ U_k &= U_k^{fix} \end{aligned} \quad (2.21)$$

- slack bus: fixed voltage magnitude and angle

$$\begin{aligned} U_k &= U_k^{fix} \\ \theta_k &= \theta_k^{fix} \end{aligned} \quad (2.22)$$

4. Solve for voltage magnitude and angle

I_{Lk} is the current of the load and I_{Gk} is the current generated at a Bus.

2.2 Approximative DC Solutions to the Power Flow Problem

In the previous chapter, we have derived the exact expression of the power flow equations. However, since solving these equations can be computationally slow and might not converge efficiently in various cases. Therefore, a linear approximation to the power flow equation can be applied to obtain a solution. In general, the approximation helps to identify critical cases, which are then analyzed further by looking at the exact expressions. They provide good intuition and are also often used to provide accurate guesses on initial conditions that can then be used to solve the complete equations.

In the DC approximation, the resistance is assumed to be zero. We assume that the lines are lossless, so there is no Ohmic loss. We have Z (impedance) composed by resistance plus reactance, and the inverse of $Z=Y$ (composed of subseptance and conductance G). For the DC approximation you assume that there is no AC part, AC is usually associated with signals that vary in time that then form the complex parts of the electrical part. In a Power System when we go to high voltages, resistance is negligible compared to the reactance which is why we can put it to 0. In Power systems, the loss due to Ohmic losses is very small.

In general, phase-shifting transformers change the angle and the magnitude. In this case, the magnitude of the voltages is equal and we only shift the angles. With the DC approximation, we assume the magnitude of the voltages to be equal to 1. The voltage difference is given by the phase shift, allowing the current to flow. As derived in the previous chapter, from the net power at the node we can get the power flows by finding first the angles and then getting the power flows. From Eq. 2.18 and Eq. 2.19 and setting G to zero we have :

$$P_k = U_k \sum_{m=1}^N U_m B_{km} \sin(\theta_{km}) \quad (2.23)$$

$$Q_k = U_k \sum_{m=1}^N U_m B_{km} \sin(\theta_{km}) \quad (2.24)$$

Additionally, the following approximations are often valid, particularly during light load conditions, i.e. small values of θ_{km} where $\sin(\theta_{km}) = \theta_{km}$ and voltage magnitude of 1:

$U_k = U_m = 1$ p.u. We also know that $B_{km} = -\frac{1}{X_{km}}$ when $R_{km} = 0$. We can write Eq. 2.23 and Eq. 2.24 in a matrix format with $\mathbf{B}$ the admittance matrix.

$$\mathbf{P} = \mathbf{B}\boldsymbol{\theta} \quad (2.25)$$

and then from the formula

$$P_{km} = X_{km}^{-1}\theta_{km} \quad (2.26)$$

we can derive the power flows.

The goal is to calculate the power flows because the power injected at the different buses is assumed to have been provided. For example, a bus is associated with a city. We know the demand (the active power) that is required for the city. We have the power generated, what we don't know is how much power we have to generate at the different buses to sustain the demand of the different cities.

Chapter 3

Graph Neural Networks

In this chapter, we will introduce the fundamentals of Graph Neural Networks to gain a better understanding of their application to the Power flow equation.

3.1 Graph Topology

A graph is a representation where entities are related to one another. For example, proteins (e.g. Fig 3.1), similarity networks, or relational structures can be represented as a graph. In this sense, a graph can be described as a common language where the choice of the nodes, edges, and topography of the graph is very important. Graph G can be written as $G = (V, E)$ where V is the set of nodes/vertices and E is a set of edges (nodes' connectivity). We will introduce the topology of graphs:

- v_i is the i^{th} node
- e_{ij} is the edge from the i^{th} node to the j^{th} node
- X is the nodal feature matrix of size $n \times f$
- A is the adjacency matrix of size $n \times n$ with entries $a_{ij} = 1$ if the nodes are connected and $a_{ij} = 0$ if not
- A degree matrix D shows the number of neighbors of each node which is assigned diagonally

There are different types of graphs :

- **Unweighted graphs:** If we care only if two nodes are connected we call such a graph unweighted graphs. For the nodes with an edge between them, we say they are adjacent or neighbors of one another.
- **Weighted graphs:** In reality, however, the edges have numerical properties that determine the relationship between two nodes. So we associate each edge e with a real value $w(e)$ that we call the weight of edge e . The way to represent weighted graphs

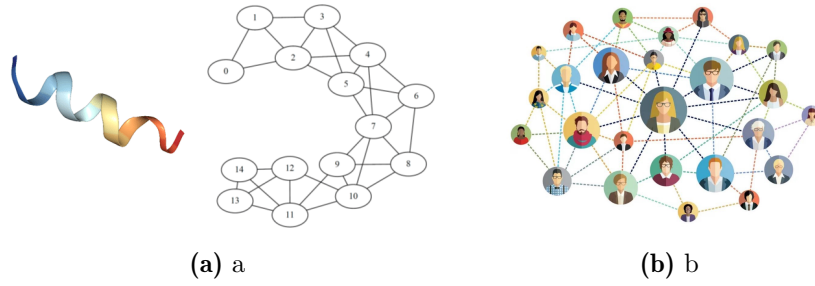


Figure 3.1: A graph for structure-based predictions of protein-protein interactions [2] and a social network [3]

is an extension of the adjacency matrix with a weight matrix such as $W=[w_{ij}]_{n \times n}$ with w_{ij} the weight of the edge between node i and j .

- **Directed Graph:** In directed graphs, the edges specify the route that must be followed to go from one linked node to another. Arrows are frequently used to depict the edges.
- **Undirected Graph:** Undirected graphs do not indicate the direction in which a path between nodes must be traveled. Instead, movement along an edge in either direction is permitted between nodes. Undirected graphs don't have loops or many edges.

3.2 Neural Networks

In traditional machine learning methods, data is used that is defined in the Euclidean space [13]. Euclidean data refers to data that can be rationally modeled as being displayed in n -dimensional linear space, such as picture files (where the x , y , and z coordinates relate to each pixel's position and the color/intensity of each pixel, respectively) [13]. Meaning only the values have significance, they are compared to one another and the ML algorithm detects patterns and similarities. However, the world usually cannot be represented in Euclidean space. Certain data such as a social network depicted by a graph do not transfer nicely into R^n . There is no clear, logical way to translate these properties to higher dimensional Euclidean space without losing information such as the quality of edges, the values associated with nodes, or the directionality of edges. Additionally, depending on the chosen embeddings, wrong correlations between data can be made (e.g. two unconnected nodes appearing closer to each other in the embedding than to nodes they are actually connected to). In order to prevent the loss of this information during the transformation of the data into a Euclidean input (as required by existing techniques), other methods have been developed. Graph Neural Networks attempt to adapt existing Machine Learning technologies to directly process non-Euclidean data as input, meaning the topology of a graph is taken into account.

We will give an introduction to fully connected Neural Networks and Convolutional Neural Networks to briefly explain the fundamentals of Neural Networks and then introduce Graph Neural Networks that are based on graph-feature learning.

3.2.1 Fully Connected Neural Network

A fully connected Neural Network is the simplest form of a Neural Network [14] where every node, or neuron, in one layer, is linked to every neuron in the subsequent layer. The input layer stores the input data and is then processed by the hidden layers. The layers are connected to one another with weights. These weights are the trainable parameters that are tuned in the model according to the loss. The input data is multiplied by the weights and then applied to a non-linear activation function (e.g. Relu, sigmoid, or tanh) to extract hidden features. The loss is determined by the difference between the predicted output data from the model and the already known output data. The computation for learning is done with backward and forward propagation [15], where the weights are constantly updated for a more accurate prediction. In general, this falls under the definition of an optimization problem that is solved with well-known algorithms such as stochastic gradient descent [16] or Adam optimizer [17].

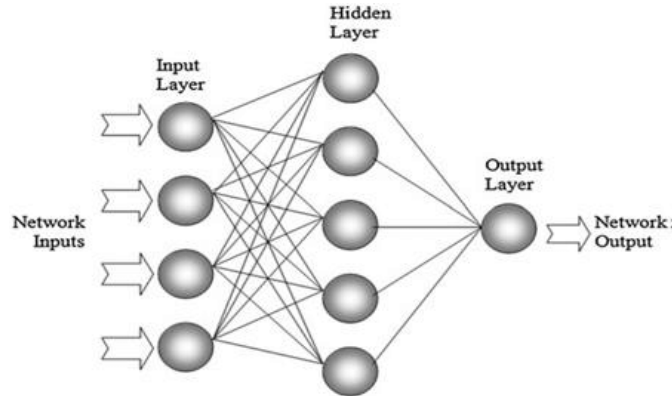


Figure 3.2: Fully connected Neural network [4]

FCNN can either be used for classification or a regression task, depending on the case. It is efficient for flat-shaped data as it is very accurate and can be a very "deep" network (many hidden layers). It is a very general model that is widely applicable as there are no assumptions that need to be made for the input. However, different data types such as grid-shaped data for image recognition or graph structures need to be flattened first where information gets lost. Hence, more structure-specific algorithms have been developed to increase performance for different data types.

3.2.2 Convolutional Neural Networks

Convolutional Neural Network (CNN) is appropriate for grid-shaped data, which is defined as a data type with fixed node ordering and size [18]. For example, CNN is often used for digital image processing where the grid data is taken out of a 2D digital image with fixed columns, rows, and node ordering. The data is initially subjected to a convolutional filter by CNN, after which its output is flattened, it is then forwarded to an FCNN layer. Flattening is the process of reducing any shape's data to a single dimension. The locality features of the grid-type data are leveraged by the convolution filter. The general architecture of a CNN is shown in Fig.3.3. Generally, after the convolutional layer, there is a pooling layer that reduces the dimension of the feature map.

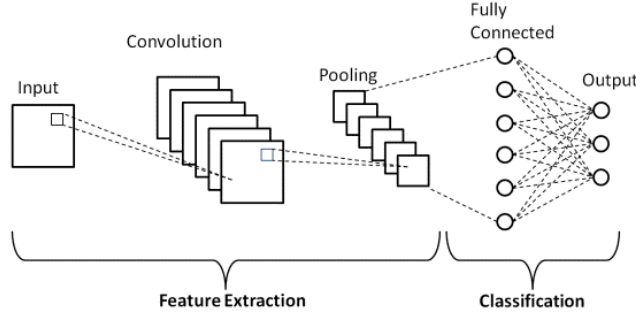


Figure 3.3: Convolutional Neural Network [5]

However, CNN can not be applied to graph data as it only works for fixed size and node ordering and graphs have no fixed node ordering. Recent developments in machine learning offer a way to embed the topology of graph data into the algorithm. This framework is called Graph Neural Network (GNN).

3.3 Graph Neural Network

Graph Neural Networks try to imitate feature extraction to graph data by implementing the graph topology (Edges, Nodes, neighbors, etc.). A single layer of a GNN has two components: Message transformation and message aggregation.

Message computation

Each node will create a message which, will be sent to other nodes later, e.g take the previous layer of embedding and multiply with weight matrix: $\mathbf{m}_u^l = \mathbf{W}^l \mathbf{h}_u^{l-1}$. We need to decide the matrix function [6].

Aggregation

Each node will aggregate the messages from node v neighbors, where the input node embedding is defined as $\mathbf{h}_v^{l-1}, \mathbf{h}_{u \in N(v)}^{l-1}$ (from the node itself + neighboring nodes).

The aggregation function is written such that it takes messages from the previous level

m and aggregates them into a single message (e.g. sum, mean or max aggregator).

$$\mathbf{h}_v^l = \text{AGG}^l(\{\mathbf{m}_u^l, u \in N(v)\}) \quad (3.1)$$

One important issue is that information from the node itself could get lost (does not directly depend on $\mathbf{h}_u^{l-1}$). Solution: Usually a different message computation will be performed. The weight matrix $\mathbf{W}$ will be applied to the neighbors u and another weight matrix $\mathbf{B}$ will be applied to the node v itself: $\mathbf{m}_u^l = \mathbf{W}^l \mathbf{h}_u^{l-1}$ and $\mathbf{m}_v^l = \mathbf{B}^l \mathbf{h}_v^{l-1}$ [6]. After aggregating from neighbors, we can aggregate the message from node v itself. First, aggregate from neighbors, and retain information, and then add retained information about the node (i) we already had.

$$\mathbf{h}_v^l = \text{CONCAT}(\text{AGG}^l(\{\mathbf{m}_u^l, u \in N(v)\}), \mathbf{m}_v^l) \quad (3.2)$$

A nonlinear activation function introduces non-linearity into the network, meaning the output cannot be reproduced by a linear combination of the inputs. It allows the stacking of multiple layers of neurons to create a deep neural network. It is often written as $\sigma(\cdot)$, $\text{ReLU}(\cdot)$, $\text{Sigmoid}(\cdot)$ and it can be added to a message or aggregation.

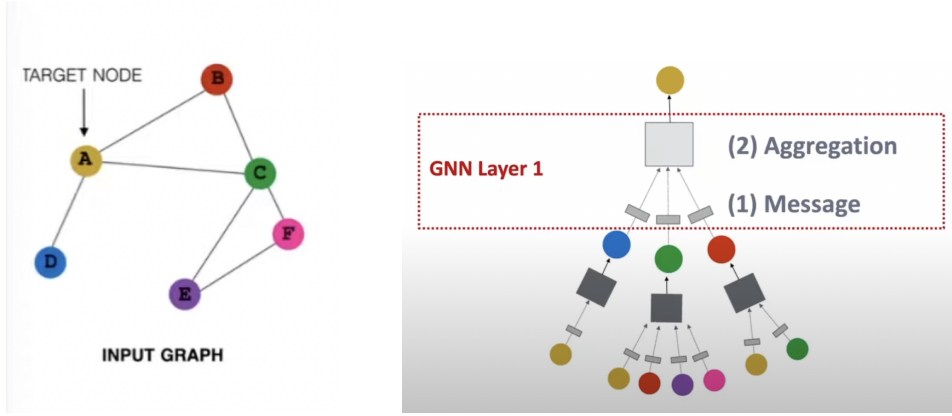


Figure 3.4: Input graph and computation graph for a target node [6]

3.3.1 Message passing layers

We will introduce a few general message-passing layers used today in most GNNs.

(1) Graph Convolutional Network (GCN)

We describe a GCN layer to show practically the functioning of the message and aggregation function. With the message equation being the weight vector multiplied by the embedding and normalized by node degree, and the aggregation function being the sum.

$$\mathbf{h}_v^l = \sigma \left(\sum_{u \in N(v)} \mathbf{W}^l \frac{\mathbf{h}_u^{l-1}}{|N(v)|} \right) \quad (3.3)$$

(2) GraphSAGE

$$\mathbf{h}_v^l = \sigma \left(\mathbf{W}^l \cdot \text{CONCAT} \left(\mathbf{h}_v^{l-1}, \text{AGG} \left(\{\mathbf{h}_u^{l-1}, u \in N(v)\} \right) \right) \right) \quad (3.4)$$

It builds upon the GCN but extends it in several aspects. The message is computed within the aggregation (allows for different aggregation methods). It takes the message from the node itself, transforms it, and then concatenates it. Two-stage approach: We take the individual messages and transform them through linear combinations. And then we apply the aggregation operator that gives a summary of the messages of the neighbors.

Stage 1

$$\mathbf{h}_{N(v)}^l = \text{AGG} \left(\{\mathbf{h}_u^{l-1}, u \in N(v)\} \right) \quad (3.5)$$

And the second step, take the messages coming from the neighbors already aggregated and concatenate them with v's own embedding from the previous layer and then multiply them with a transformation matrix and then pass through a nonlinear equation.

Stage 2

$$\mathbf{h}_v^l = \sigma \left(\mathbf{W}^l \cdot \text{CONCAT} \left(\mathbf{h}_v^{l-1}, \mathbf{h}_{N(v)}^l \right) \right) \quad (3.6)$$

What kind of aggregation can be used?

Mean : Takes a weighted average of neighbors

Pool : Transforms neighbor vectors and applies symmetric vector function $\text{Mean}(\cdot)$ or $\text{Max}(\cdot)$.

GraphSage adds the notion of l_2 normalization that is the "the normalization technique that modifies the dataset values in a way that in each row the sum of the squares will always be up to 1" [19]. Without l_2 normalization the embedding vectors can have different scales. In some case l_2 normalization result in better performance

(3) Graph Attention Network

With every neighbor we have an associated weight. This weight we call attention weight is written as α_{vu} . This weight provides the importance of a neighbor of a given node. So messages coming from different nodes will have different weights. In GCN, there is already an implicit notion of α_{vu} that was only dependent on the degree of node v and is explicitly based on the structural properties of the graph. In graph sage all neighbors are equally important to node v while for GATs the attention α_{vu} focuses on the important parts of the input data and fades out the rest so that the NN devotes more computing power on the important part of data. We are implicitly specifying different weights to different nodes in a neighborhood. With the Graph Attention Network we are looking if we can do better than with simple neighborhood aggregation. The idea is to compute embedding $\mathbf{h}_v^l$ of each node in the graph following an attention strategy:

(1) Let a function a compute attention coefficients e_{vu} across pairs of nodes u, v based on their messages:

$$e_{vu} = a(\mathbf{W}^l \mathbf{h}_u^{l-1}, \mathbf{W}^l \mathbf{h}_v^{l-1}) \quad (3.7)$$

where e_{vu} indicates the importance of u 's message to node v . The function a can be chosen arbitrarily, in most cases a simple single layer neural network is used. The parameters of a are learning simultaneously to the weight matrices.

(2) Normalize the attention coefficient to obtain the final attention weight α_{vu} :

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})} \quad (3.8)$$

For the message aggregation a weighted sum is done based on the final attention weight:

$$\mathbf{h}_v^l = \sigma \left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \right) \quad (3.9)$$

Multi-head attention: Stabilizes the learning process of attention mechanism. The idea is to create multiple attention scores (each replica with a different set of parameters). The outputs are then aggregated by concatenation or summation which can be seen in Fig. 3.5. This allows for a more robust model and it reduces the risk that the model will get stuck in some optimization space. Each of the attention scores have to be predicted with a different function a and each of these functions a have to be initialized with a different set of random parameters [6].

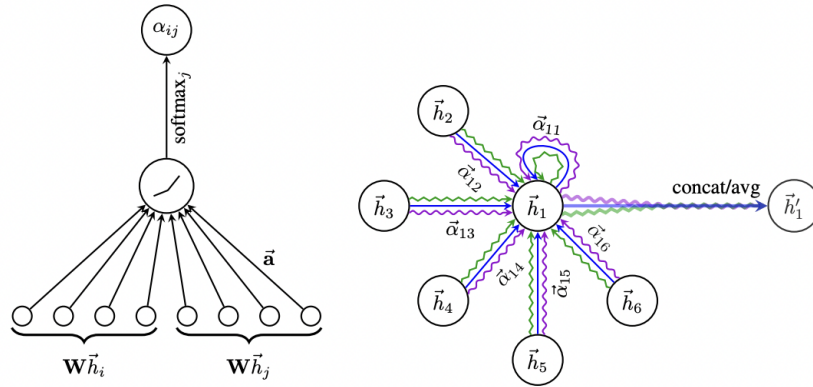


Figure 3.5: Attention mechanism(left) and multi-head attention with $K=3$ heads by node 1 of its neighborhood. Different arrow styles show different computation methods [7]

The benefit is that this allows to specify different importance values to different neighbors, it is computationally efficient (it can be paralyzed across all the incoming messages), it is storage efficient and it has a fixed number of parameters (independent

of graph size). It is also practical as it can only look at a defined specific neighborhood and it not dependent on the global graph structure. Meaning that the functions used are transferable for different type of graph structures [6]

Chapter 4

Graph Neural Network for Power Flow

In chapter 1 we have explained the fundamentals of Power Systems and derived the Power Flow equations. In chapter 2 we have introduced the basics behind Graph Neural Networks and its architecture. In this chapter we will introduce our novel method to use Graph Neural Networks to compute solutions to the Power Flow equations. The graph topology of an electrical transmission grid will be integrated in our Neural network and the physical data from the nodes and edges of the graph (active and reactive power and voltage magnitude and angle) will be used as input and output data. We have based our approach on previous research, especially [20] that have employed Graph Neural Networks applied to power flows.

Firstly, we will use traditional methods to generate the features of a 24-Bus system, by using a Matlab library: Matpower [21]. Secondly, we have designed a GNN that is able to predict the output values needed for Power Flow computation. We will walk through the algorithm, by explaining the grid architecture of a 24-Bus System, the methodology for data processing used, and the message passing layer and GNN architecture chosen. Lastly, we will present our results by evaluating the accuracy of the predictions against our original dataset.

4.1 Traditional method

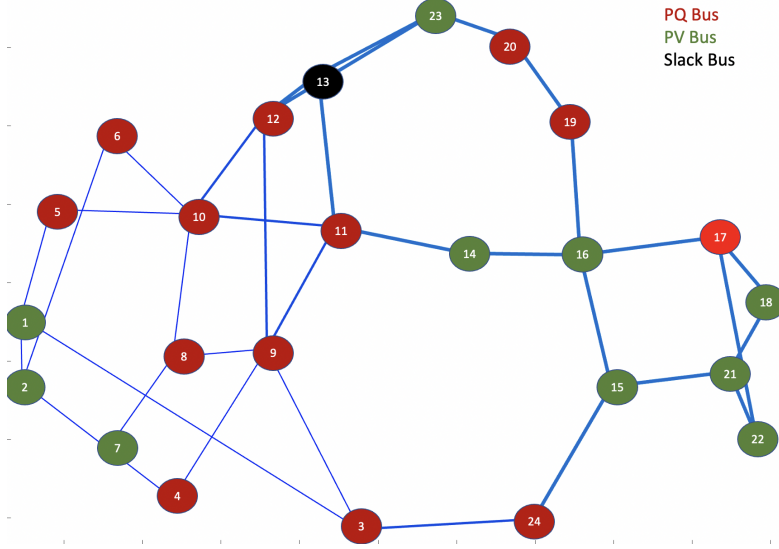


Figure 4.1: Graph representation of a 24-Bus electrical power grid

Fig. 4.1 illustrates an example of a graph representation of a 24-Bus electrical power grid. The types of buses explained in chapter 1 are represented by different colors (PQ: red, PV: green, Slack: black). At each bus four variables are calculated: active power (P), reactive power (Q), voltage (V) and voltage angle (θ). Looking at Eq. 2.18 and Eq. 2.19, we can see that the equation system can be solved for each one of these values based on the known and unknown variables at each bus. From the table in Fig.2.2 we have shown the different known variables. The computation of these variables is important, for example to predict how much power is necessary to be generated to satisfy the demand in the grid. To compute these values, traditionally an iterative technique such as the Newton Raphson [8] method is used. There are several computations that can be made:

- **Optimal Power Flow:** This is an optimization problem. It is evaluated by wanting to minimize the cost that each generator is subjected to, to generate the inquired power. For the optimal Power Flow the most important constraints are to respect demand (generated power) and the technical constraints to the lines (Power Flow sustained by a certain line is limited dependent on material used) [22].
- **AC Power Flow:** The ACPF calculation is the nonlinear solution to the equations we derived in the chapter 1.
- **DC Power Flow:** DCPF calculation takes several assumptions that simplify the ACPF equations. We assume that the magnitude of the voltages are equal to 1 and that the angle difference between neighboring nodes are small. This gives us the approximation described with Eq. 2.26.

With the Matlab library MatPower [21], it is possible to generate these variables. We were able to extract 5000 datapoints (Power demand at each hour for 5000 hours) and to store the important data in an excel file. We were also able to extract an Edge index list, showing the grid topology and more specifically which nodes are connected to one another. Hence, each node has the following features:

- Type of node with labels : 1=PQ,2=PV,3=slack
- Net real power (P)
- Net reactive power (Q)
- voltage magnitude(V)
- voltage angle (θ)

To generate the dataset we used following code:

```

1  angle=[];
2  magnitude=[];
3  TotalDC=[];
4  tic
5  define_constants;
6  mpctry = loadcase('System');
7  for j=1:5000
8      for i=1:24
9          mpctry.bus(i, PD)= hourlyDemandBus(i,j);
10     end
11     resu=mpctry.bus(:,PD);
12     case24new=runpf(mpctry);
13     angle=case24new.bus(:,9);
14     mag=case24new.bus(:,8);
15     G_index=case24new.gen(:,1);
16     GenActivePower=case24new.gen(:,2);
17     result_GenActivePower=[accumarray(G_index,GenActivePower);0];
18     GenReactivePower=case24new.gen(:,3);
19     result_GenReactivePower=[accumarray(G_index,GenReactivePower);0];
20     LoadActivePower=case24new.bus(:,3);
21     LoadReactivePower=case24new.bus(:,4);
22     NetREALPower=[abs(result_GenActivePower-LoadActivePower)];
23     NetReactivePower=[abs(result_GenReactivePower-LoadReactivePower)];
24     edgeindex=[case24new.branch(:,1),case24new.branch(:,2)];
25     fill=[];
26     for k=1:24
27         matrix=[NetREALPower(k),NetReactivePower(k),mag(k),angle(k)];
28         fill=[fill,matrix];
29     end
30     TotalDC=[TotalDC;fill];
31 end
32 %%%%%%%%%add label 1:PQ, 2:PV, 3: ref
33 Bustype=case24new.bus(:,2);
34 totbustype=[];
```

```

35 for j=1:24
36     matrixbus=[Bustype(j),Bustype(j),Bustype(j),Bustype(j)];
37     totbustype=[totbustype,matrixbus];
38 end
39 %%%%%%%%% add legend P,Q,V,A
40 full={};
41 labelP=sprintfc(['P%d'], 1:24);
42 listP=[labelP];
43 labelQ=sprintfc(['Q%d'], 1:24);
44 listQ=[labelQ];
45 labelV=sprintfc(['V%d'], 1:24);
46 listV=[labelV];
47 labelA=sprintfc(['A%d'], 1:24);
48 listA=[labelA];
49 for i=1:24
50     list=[listP(i),listQ(i),listV(i),listA(i)];
51     full=[full,list];
52 end
53 %%%%%%%%%
54 full;
55 strfull=string(full); %cell array needs to be converted in string array
56 Total=[strfull;totbustype;TotalDC];
57 toc
58 t=toc
59 writematrix(edgeindex, '/path/Matlab/Case24/BIGEdgeIndex.xlsx')
60 writematrix(Total, 'path/Matlab/Case24/BIGcasenew24AC_TRYnonlabeled.xlsx')
61
62 %end

```

We are looking to calculate the ACPF, considering that our novel machine learning method for solving Power Flows relies on accurate data on the nodes and the grid. The DCPF would give us too little input data (since magnitude and reactive power are constant or equal to 0) to properly calculate the power flows. The hourly demanded file called in line 9 of the matlab code, is a file that gives the amount of power demanded at each bus for each hour for a 24-Bus electrical grid.

4.2 Power Flow computation with GNN

The goal is to develop a GNN able to predict the unknown physical variables at a node given information about its grid connectivity and input features.

4.2.1 Dataset

From our dataset generated with Matpower, we can extract our input and output features. First of all, we have to detect the different type of nodes that have different physical values as input and output. For this we loop over our labels and store the corresponding input and output features in x and y dataframe. We only have to slice our dataset in a training, validation and testing dataset. As for any regression problem, we have to normalize our data. In our case, we chose to normalize each variable

(P_i, Q_i, V_i, A_i) for all i 's per hour where i is the node. The mean and standard deviation values calculated should be stored, as they will be valuable for de-normalization when doing the predictions for the test set. The procedure for data pre-processing is the following:

- Load dataset generated with matpower
- Normalize dataset
- Split dataset in training, validation and testing
- Separate input and output features
- load Edge Index list
- Create data loader for training, validation, and test set

Hence, for the ACPF of a 24-Bus system we have following data:

- 3 input features: label (node type) and the two known variables for each node taken from the table in Fig. 2.2.
- 2 output features: two physical unknowns for each node
- Edge Index list: connectivity of nodes. The connectivity in the graph is undirected (meaning it works in both directions).

For simplicity, in our model we disregard the features of the lines/edges such as electrical current.

4.2.2 Model Architecture

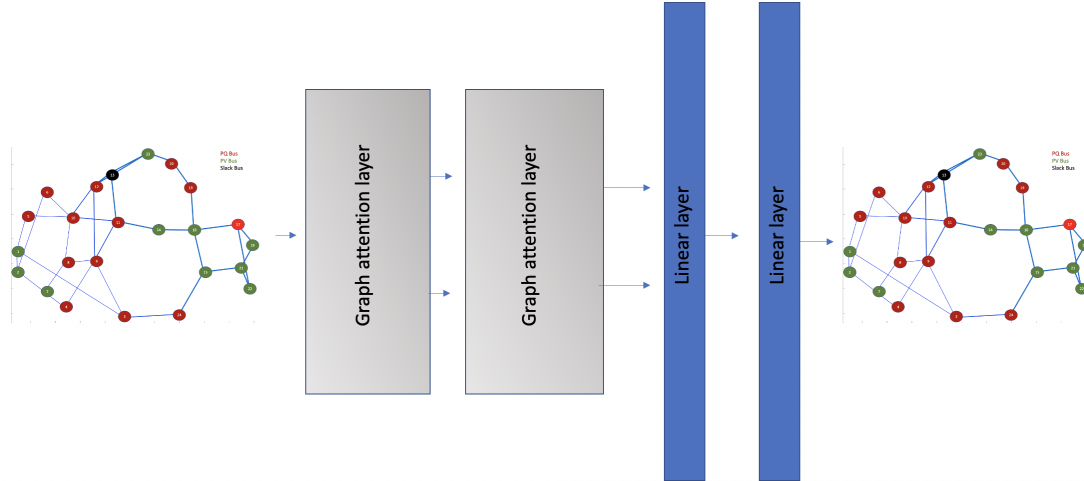


Figure 4.2: Model Architecture: Two GAT layers and two FCNN layers

For our model we have chosen the GAT message passing layer. Intuitively, the GAT framework explained in the previous chapter seems to be the most intuitive framework for representing the dynamics of an electrical power grid. It emphasizes on the relevance of the different weight of messages coming from neighboring nodes, which is a good analogy of the power flow between nodes. For this, we added 4 attention heads to our GAT layer. We decided to stack two GAT layers sequentially with the same amount of nodes. Additionally, we added two fully connected linear layers taking in the embeddings from the GAT layers as an input and outputting the two unknown features per node. For better visualization of the model architecture chosen, we have added a python snippet of our code:

```

1 class My_GNN_NN(torch.nn.Module):
2     def __init__(self, node_size, feat_in, feat_size1, hidden_size1):
3         super(My_GNN_NN, self).__init__()
4         self.feat_in = feat_in
5         self.feat_size1 = feat_in
6         self.hidden_size1 = hidden_size1
7         n_att_h = 4
8         self.conv1 = GATConv(feat_in, hidden_size1, heads= n_att_h)
9         self.conv2 = GATConv(hidden_size1*n_att_h, ...,
                                hidden_size1*n_att_h)
10        self.lin1 = Linear(hidden_size1*n_att_h, hidden_size1*n_att_h)
11        self.lin2 = Linear(hidden_size1*n_att_h, feat_size1)
12    def forward(self, data):

```

```

13         x, edge_index = data.x, data.edge_index
14         x = self.conv1(x, edge_index)
15         x = torch.relu(x)
16         x = self.conv2(x, edge_index)
17         x = torch.relu(x)
18         x = self.lin1(x)
19         x = torch.relu(x)
20         x = self.lin2(x)
21         return x
22     def save_weights(self, model, name):
23         torch.save(model, name)

```

The complete code can be seen on github (<https://github.com/LouisaHil/Graph-Neural-Network-solver-for-Power-Flow>).

4.3 Results

4.3.1 Loss and Predictions

After applying several different learning strategies and tuning of different parameters to achieve the best train and validation loss, we were able to generate good predictions of P, Q, V, and A. We focused our hyper parameter tuning on following variables:

- epochs (best epoch=230)
- batch size (=train: 10, test:3)
- learning parameter (=0.0001)
- hidden layers (=32)
- weight decay (=1e-5)
- number of attention heads (=4)

Once we achieved a satisfying train loss (approximately $10e - 5$), we saved our model in a pytorch file and loaded it for testing. After testing we denormalized our predictions, to obtain the real physical values. On the graph in Fig. 4.3 we have illustrated the train loss respective to the number of epochs. We chose the Mean Square Error function to compute our loss (MSE). The train loss is declining quickly and converging to 0. With the table in Fig. 4.3 we are showing the decay of the train and validation loss for each 20 epochs. From this we can see that our best train loss is for 230 epochs and our best validation loss for 200 epochs. However, we choose to save our model at 230 epochs.

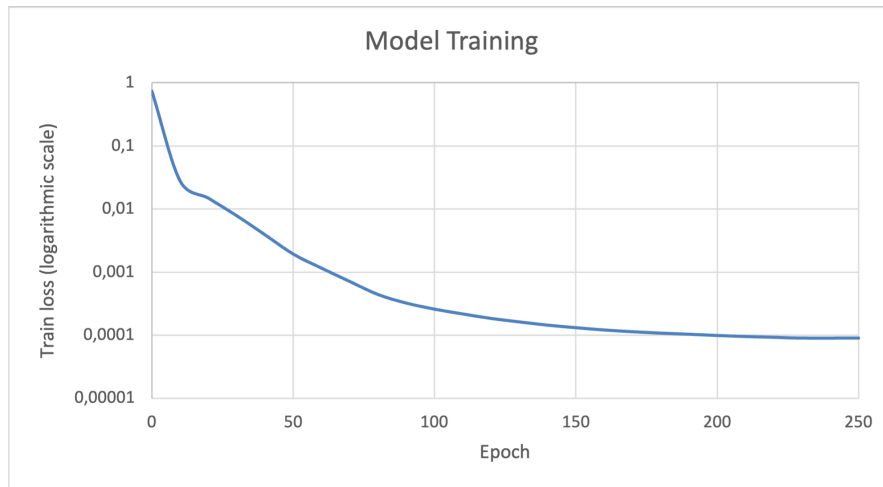


Figure 4.3: Graph of train loss (logarithmic scale) for GNN model

Looking at our training and validation loss we have the best results for:

- Best epoch: 230, train loss: $9.02e-5$ and validation loss: $6.00e-7$

We save the model with the trained parameters for the best training loss and load it for the test set. This results in a test loss of : $1.30e-6$. On the following tables in Fig. 4.4 and Fig. 4.5 and Fig. 4.6 we are showing the denormalized real predicted values of our GNN at each node for a graph at a random hour. We are showing the predictions for 3 graphs to properly depict the accuracy of our model.

Hour 1		Output (depending on Node type)			
Node	Type	Prediction	Real	Prediction	Real
1	PV (y=Q,A)	7,993	8,489	21,452	21,545
2	PV (y=Q,A)	7,904	8,618	21,352	21,393
3	PQ (y=V,A)	0,992	0,992	22,645	22,709
4	PQ (y=V,A)	0,994	0,994	16,579	16,71
5	PQ (y=V,A)	1,014	1,015	16,386	16,666
6	PQ (y=V,A)	1,01	1,01	13,542	13,383
7	PV (y=Q,A)	16,84	16,855	24,861	24,934
8	PQ (y=V,A)	0,992	0,992	18,755	18,788
9	PQ (y=V,A)	0,985	0,985	14,573	14,59
10	PQ (y=V,A)	1,017	1,017	12,835	12,953
11	PQ (y=V,A)	0,979	0,979	12,934	12,885
12	PQ (y=V,A)	0,997	0,997	11,024	10,997
13	Slack (y=P,Q)	1364,095	1366,356	481,461	481,568
14	PV (y=Q,A)	9,144	8,651	23,854	23,782
15	PV (y=Q,A)	82,604	83,076	38,916	38,884
16	PV (y=Q,A)	90,495	90,726	35,68	35,633
17	PQ (y=V,A)	1,037	1,037	42,087	42,088
18	PV (y=Q,A)	63,568	63,111	44,483	44,451
19	PQ (y=V,A)	1,022	1,022	31,012	30,979
20	PQ (y=V,A)	1,038	1,038	27,917	27,939
21	PV (y=Q,A)	109,844	109,594	44,949	44,958
22	PV (y=Q,A)	27,149	27,72	50,251	50,341
23	PV (y=Q,A)	271,738	272,679	26,612	26,654
24	PQ (y=V,A)	0,98	0,98	32,959	32,96

Figure 4.4: Predicted against real output values for hour 1

Hour 9		P,Q,V,A (depending on Node type)			
Node	Type	Prediction	Real	Prediction	Real
1	PV (y=Q,A)	7,354	8,216	21,777	21,892
2	PV (y=Q,A)	7,287	8,285	21,701	21,74
3	PQ (y=V,A)	0,992	0,992	22,973	23,053
4	PQ (y=V,A)	0,993	0,993	16,883	17,031
5	PQ (y=V,A)	1,014	1,014	16,696	16,989
6	PQ (y=V,A)	1,01	1,01	13,887	13,697
7	PV (y=Q,A)	16,944	17,115	25,262	25,324
8	PQ (y=V,A)	0,992	0,992	19,118	19,151
9	PQ (y=V,A)	0,985	0,985	14,831	14,859
10	PQ (y=V,A)	1,016	1,016	13,112	13,227
11	PQ (y=V,A)	0,979	0,978	13,107	13,07
12	PQ (y=V,A)	0,996	0,996	11,163	11,15
13	Slack (y=P,Q)	1378,49	1380,899	489,559	489,848
14	PV (y=Q,A)	10,624	10,556	24,129	24,045
15	PV (y=Q,A)	82,722	83,066	39,251	39,217
16	PV (y=Q,A)	91,805	91,926	35,98	35,941
17	PQ (y=V,A)	1,037	1,037	42,422	42,42
18	PV (y=Q,A)	63,6	63,062	44,824	44,79
19	PQ (y=V,A)	1,022	1,022	31,286	31,25
20	PQ (y=V,A)	1,038	1,038	28,15	28,165
21	PV (y=Q,A)	109,9	109,631	45,319	45,297
22	PV (y=Q,A)	27,351	27,693	50,585	50,677
23	PV (y=Q,A)	274,761	275,726	26,816	26,852
24	PQ (y=V,A)	0,98	0,98	33,297	33,299

Figure 4.5: Predicted against real output values for hour 9

Hour 28		P,Q,V,A (depending on Node type)			
Node	Type	Prediction	Real	Prediction	Real
1	PV (y=Q,A)	4,423	6,637	23,476	23,724
2	PV (y=Q,A)	4,478	6,384	23,528	23,568
3	PQ (y=V,A)	0,99	0,991	24,706	24,869
4	PQ (y=V,A)	0,992	0,992	18,5	18,724
5	PQ (y=V,A)	1,012	1,013	18,328	18,694
6	PQ (y=V,A)	1,008	1,008	15,693	15,35
7	PV (y=Q,A)	18,009	18,637	27,333	27,38
8	PQ (y=V,A)	0,99	0,99	21,019	21,066
9	PQ (y=V,A)	0,982	0,981	16,204	16,278
10	PQ (y=V,A)	1,013	1,013	14,55	14,67
11	PQ (y=V,A)	0,976	0,976	14,082	14,043
12	PQ (y=V,A)	0,994	0,994	12,034	11,957
13	Slack (y=P,Q)	1454,811	1456,868	534,189	534,754
14	PV (y=Q,A)	19,91	20,965	25,571	25,436
15	PV (y=Q,A)	83,299	82,945	41,003	40,972
16	PV (y=Q,A)	98,439	98,39	37,596	37,56
17	PQ (y=V,A)	1,037	1,0364	44,205	44,164
18	PV (y=Q,A)	63,976	62,82	46,602	46,601
19	PQ (y=V,A)	1,022	1,022	32,677	32,671
20	PQ (y=V,A)	1,038	1,038	29,362	29,356
21	PV (y=Q,A)	110,549	109,819	47,205	47,085
22	PV (y=Q,A)	29,163	27,555	52,341	52,448
23	PV (y=Q,A)	291,232	292,315	27,838	27,896
24	PQ (y=V,A)	0,98	0,98	35,131	35,081

Figure 4.6: Predicted against real output values for hour 28

Comparing the real values with the predicted values we can say that our model gives a sufficiently good prediction. For better visualization, we rounded the values at 3 significant figures after the comma. Looking at further data, we observe that the result for the first two nodes are most likely to have bigger deviations than the rest of the nodes. However, no clear deduction can be made from this observation. We can also observe that the predictions are slightly better for PQ Buses than for PV Buses. However, there is no clear explanation on why this happens. For further analysis, the excel file with the predictions and loss can be generated from the code that is accessible on github: <https://github.com/LouisaHil/Graph-Neural-Network-solver-for-Power-Flow>. The resulting excel files from the code are also uploaded in the folder "Results". For further information on the code, please look at the "Read me" file.

Concluding from our predictions, our novel GNN solver is a very promising solution to Power Flows. For even better predictions, we could tune further the hyperparameters (e.g. increase number of attention heads), add edge features such as electrical current or change the GNN architecture specifically the GNN message passing layer.

4.3.2 Computation time

In praxis, the main motivation behind finding a new solver for the power flow is mainly due to the need of quick computation so that the electricity suppliers can respond in a timely manner to transmission line failures. We will therefore evaluate the computation time of the traditional method with Matpower against our GNN method. For both

algorithm we compute the output for 5000 graphs of 24 nodes. Of course, we only measure the time taken by our predicting algorithm of our GNN.

To obtain the results of our GNN we ran our codes (Matlab and Python) on the ETH Euler cluster. Here, it is important to chose the same computation node of the cluster so that we can compare the times. We chose the node *EPYC7742* and we run the code with following commands:

GNN model on Python

```
1 $ sbatch --constraint=EPYC_7742 --mem-per-cpu=5000 --wrap "python ...
    mainGNN.py"
```

Here, the time taken depends a little bit on the batch size chosen. We did several computations and observed the quickest computation with a batch size of 20. Of course here, we also need to look at the test loss, since the bigger the batch size, the higher the test loss. We measured a computation time of 1.77 minutes with a test loss of $3.0E-7$ for a batch size of 20 as seen on the image in Fig. 4.7.

```
For a Dataset of 5000 hours we have:
batch size is : 20
=====
elapsed time (in minutes) is : 1.7694368362426758
=====
test loss: 0.0000003
=====
```

Figure 4.7: Batch size, computation time and test loss for predictions with GNN model

Traditional solver with Matpower on Matlab:

```
1 $ sbatch --constraint=EPYC_7742 --mem-per-cpu=1024 --wrap "matlab -r ...
    case24try"
```

Here, it is important to load the Matpower folder onto the cluster and to add the paths to the folder in the Matlab file (see Matlab code). As seen on the image in Fig. 4.8, we measure a time taken of 35.56 seconds.

```
Elapsed time is 35.558003 seconds.

t =

    35.5588
```

Figure 4.8: Computation time with Matpower

Concluding, the traditional solver with Matpower is still almost 3 times faster than the GNN method. However, considering that we only tested on a dataset of one size

(5000 hours) and the computation time of the Matpower solver is not necessarily linear, the GNN might perform better for a bigger dataset (e.g bigger power network).

The GNN is also running on a CPU (Euler cluster) and not on GPU. GNN are known for performing especially well on a GPU. Therefore, in order to have a proper understanding for the computation time of our model we should run it on GPU node (which we don't have access to) .

Chapter 5

Conclusions and Outlook

In this thesis, we have investigated the adequacy of a Graph Neural Network solver used for Power Flow computation. In the first section, we show the derivation of the Power Flow equations for a transmission line and for node networks. This analysis was crucial to understand the behavior and relationships between Power, Voltage, and Current in a network. In the second part, we introduced Graph Neural Networks by highlighting the terminology of graph theory and by introducing Neural Networks and the rise of Graph Neural Networks. We show examples of the most used GNN message-passing layers and explained their action principles. Lastly, based on this theory, we designed a Graph Neural Network aiming to solve the power flow equations. We extracted data for a 24-Bus electrical grid generated with the matpower library. We implemented the graph topology of this 24-bus grid into our GNN and used the known variables for every node as input for training. As a result, we obtained very promising results and were able to compute the desired output features for every bus. However, the GNN method still had a larger computation time compared to the traditional solution with Matpower. This is a problem that can be solved by running the GNN on a GPU node.

For future research, it would make sense to develop a custom-made message-passing layer that incorporates the physics of the electrical grid. One could also look at increasing the number of features chosen as input. For example, from the edges/lines of a grid, we could take into account the current. It would also be interesting to see the performance of this solution on Optimal Power Flow. This computation would be beneficial in determining the power generated in total to sustain the demand for every bus. Since OPF is an optimization problem, it should be easily transferable to machine learning methods.

The increase in performance of the machine learning model would be very valuable to improve the low computational efficiency of today's power flow solvers. The method adopted with GNNs makes it possible to perform a load study in real-time and is therefore an important asset to the development of smart grids and sustainable and resilient electricity distribution.

Bibliography

- [1] G. Hug, “Power System Analysis,” *Lecture 227-0526-00, ITET ETH Zurich*, September 2021.
- [2] M. Baranwal, A. Magner, J. Saldinger, E. Turali-Emre, S. Kozarekar, P. Elvati, J. VanEpps, N. Kotov, A. Violi, and A. Hero, “Struct2Graph: A graph attention network for structure-based predictions of protein-protein interactions,” *BMC Bioinformatics*, 09 2020.
- [3] S. Nanawati, “Social Network Analytics,” *Medium-Analytics Vidhya*, 12 2021.
- [4] A. Majidi, G. R. Lashkaripour, Z. Shoaie, M. Noruzi, and Y. Firouzei, “Estimating compaction parameters of marl soils using multi-layer perceptron neural networks,” vol. 20, pp. 170–198, 01 2014.
- [5] S. Balaji, “Binary Image classifier CNN using TensorFlow,” *Medium-Techiepedia*, 12 2021.
- [6] Jure Leskovec, “CS224W: Machine Learning with Graphs | 2021 | Lecture 7.1 - A general Perspective on GNNs,” 5 2021.
- [7] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph Attention Networks,” *arXiv*, 2017.
- [8] Wikipedia, “Newton’s method.” [Online, August 2014].
- [9] Wikipedia, “Telegrapher’s equations..” [Online, August 2021].
- [10] Wikipedia, “Lumped-element model.” [Online, September 2022].
- [11] M. Bolotinha, “What is Shunt Reactor: Types, Construction and Applications,” *Electrical technology*, 8 2018.
- [12] Wikipedia, “Ohm’s law.” [Online, September 2022].
- [13] Wikipedia, “Euclidean space.” [Online, December 2022].
- [14] C. Y. S. W. Zahangir Alom, Tarek Taha, “A State-of-the-Art Survey on Deep Learning Theory and Architectures,” *Electronics*, March 2019.

- [15] H. J. Kelley, "Gradient theory of optimal flight paths," *Ars Journal*, vol. 30, no. 10, pp. 947–954, 1960.
- [16] Wikipedia, "Stochastic gradient descent." [Online, November 2022].
- [17] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv*, 2014.
- [18] Wikipedia, "Convolutional neural network." [Online, August 2014].
- [19] T. van Laarhoven, "L2 Regularization versus Batch and Weight Normalization," *arXiv*, 2017.
- [20] M. S. M.G.A. Fatah, Bert J. Claessens, "Integrating Power Grid Topology in Graph Neural Networks for Power Flow," *Technische Universiteit Eindhoven*, May 2022.
- [21] C. E. M.-S. R. D. Zimmerman and R. J. Thomas, "MATPOWER: Steady-State Operations, Planning and Analysis Tools for Power Systems Research and Education," *Power Systems, IEEE Transactions on*, vol. 26, no. 1, pp. 12:19, Feb 2011.
- [22] M. R. AlRashidi and M. E. El-Hawary, "Applications of computational intelligence techniques for solving the revived optimal power flow problem," *Electric Power Systems Research*, 79, 2009.